MODULE

09

Git internals

Open the hood: objects, trees, refs and HEAD. After this, Git stops being magic.

YOU WILL LEARN

Objects	`01_objects.md`
Refs & HEAD	`02_refs_and_head.md`
A commit by hand	`03_commit_by_hand.md`
Exercise: plumbing commit	`ex09_commit_by_hand/`
Interview Q&A	`INTERVIEW.md`

WHY INTERNALS

Why look inside .git?

THE PROBLEM

Why does a rebase change commit IDs? Why is creating a branch instant? How can reflog bring back ”deleted” commits? Memorising commands without the model behind them makes Git feel random.

THE IDEA

Git is a small database of objects plus name tags (refs). Learn the 4 object types and everything else follows.

PICTURE IT OBJECTS

Four kinds of objects

📚 A library where every item is filed under a **fingerprint** (hash) of its content: identical content gets the same fingerprint and is stored once. Change one letter and you get a new fingerprint.

`blob`
file CONTENT (no name)

→

`tree`
a folder: names → blobs / trees

→

`commit`
tree + parent(s) + author + message

→

`tag`
annotated name for an object

TRY IT OBJECTS

Walk from a commit down to a blob

```
$ git cat-file -p HEAD
tree 362d89e03cdf686deaa193f9f07649a31e67343b
author Vishal <vishal@example.com> 1793088180 +0000
committer Vishal <vishal@example.com> 1793088180 +0000

First snapshot
$ git cat-file -p HEAD^{tree}
100644 blob ce013625030ba8dba906f756967f9e9ca394464a    a.txt
100644 blob ce013625030ba8dba906f756967f9e9ca394464a    copy-of-
a.txt
040000 tree 5234dadb279379214f5a218b331d5db0a198db4f    src
$ git ls-tree -r HEAD
100644 blob ce013625030ba8dba906f756967f9e9ca394464a    a.txt
100644 blob ce013625030ba8dba906f756967f9e9ca394464a    copy-of-
a.txt
100644 blob 9cf595fed1e892474f91a6d40ebb96d21c66b89d    src/app.py
$ git cat-file -p HEAD:a.txt
hello
```

1. The commit points to ONE tree (the snapshot).
2. Two files, same content: **one blob**.
3. Snapshots, not diffs; unchanged files are reused.

STEP BY STEP HASHES

Why IDs change after rebase and amend

A COMMIT'S ID IS A HASH OF...	CHANGED BY
`its tree (the snapshot)`	any file change
`its parent ID(s)`	**rebase** (new parent)
`author + date`	--reset-author, rebasing (committer date)
`the message`	**amend**, reword

> Change anything → a new hash → a new commit. That's why rewritten history is "different" history.

TRY IT REFS AND HEAD

Branches and HEAD are tiny text files

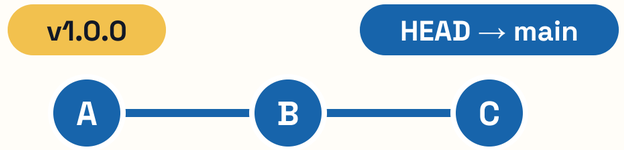
```
$ cat .git/HEAD
ref: refs/heads/main
$ cat .git/refs/heads/main
bcf013a339ffca30ac033dc317615cef294a4821
$ git show-ref
bcf013a339ffca30ac033dc317615cef294a4821 refs/heads/main
bcf013a339ffca30ac033dc317615cef294a4821 refs/tags/v1.0.0
$ git switch --detach HEAD~0
HEAD is now at bcf013a Version 1
$ cat .git/HEAD
bcf013a339ffca30ac033dc317615cef294a4821
```

1. A branch = a file with one commit ID.
2. HEAD = ”ref: refs/heads/main” (symbolic).
3. Detached HEAD = HEAD holds a commit ID directly.

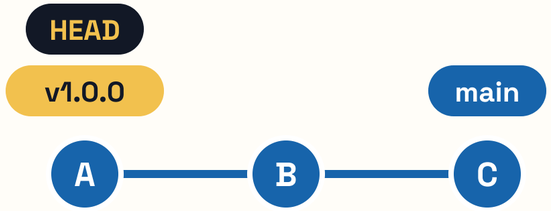
PICTURE IT HEAD

Attached vs detached HEAD

Normal: HEAD → main



`git switch --detach v1.0.0`



Detached: HEAD points at a **commit**, not a branch. New commits here belong to no branch, so create one with **git switch -c**.

STEP BY STEP REFS AND HEAD

Naming commits

EXPRESSION	MEANS
`HEAD~1` = `HEAD~` = `HEAD^`	the (first) parent
`HEAD~3`	three generations back (first parents)
`HEAD^2`	the SECOND parent of a merge commit
`main@{1}`	where main was one move ago (reflog)
`v1.0.0^{tree}`	the tree of the commit a tag points to
`HEAD:src/app.py`	a file's blob in a commit

TRY IT PLUMBING

A commit built by hand

```
$ git hash-object -w hand.txt
1d00f59d814e68d3cbf6f8b74793500b5e916ed0
$ git write-tree
6090a5336cb98585d4646beed4c54b9438588a9f
$ git commit-tree 6090a53 -p HEAD -m "Built by hand"
5d86025e830748472b7b7e575f01623a2c168b02
```

1. **hash-object -w**: store the content → blob.
2. **update-index** + **write-tree**: index → tree.
3. **commit-tree** → commit; **update-ref** moves the branch.

COMMON MISTAKE DETACHED HEAD

Committing in detached HEAD, then leaving

✗ WRONG

```
git switch --detach v1.0.0
# ...two commits of experiments...
git switch main
# warning: you are leaving 2 commits behind
```

Those commits belong to no branch. They're not deleted yet, but they're easy to lose and will be garbage-collected.

✓ RIGHT

```
git switch --detach v1.0.0
# ...commits...
git switch -c experiment     # keep them on a branch
# already left? git reflog -> git branch rescue <id>
```

Create a branch before leaving, or rescue the commits with the reflog.

IN REAL JOBS INTERNALS

Where internals help at work

Scripts and CI

```
git rev-parse --short HEAD   # image tag
git rev-parse --abbrev-ref HEAD
```

Inspect without checking out

```
git show v1.2.0:config/settings.py
git ls-tree -r --name-only v1.2.0
```

Repo size problems

```
git count-objects -vH
git gc
```

Understand ”weird” history

```
git cat-file -p <merge-commit>  # 2 parents
```

INTERVIEW SAY THIS IN THE INTERVIEW

“How does Git store data internally?”

- A content-addressable **object database**: blobs (content), trees (folders), commits, annotated tags.
- Each object is named by the **hash of its content**; identical content is stored once.
- Commits are **snapshots**: a tree + parent(s) + metadata.
- Branches, tags and HEAD are **refs**: tiny files pointing at object IDs.
- That’s why branches are cheap and rebased commits get new IDs.

> **Follow-up they ask next:** ”What is a detached HEAD?” (HEAD points at a commit, not a branch; create a branch to keep new commits).

RECAP REMEMBER THESE

Module 9 in 5 lines

1. 4 objects: **blob**, **tree**, **commit**, **tag**, named by content hash.
2. Commits are snapshots that point to a tree and parent(s).
3. Refs (branches, tags) are files with IDs; HEAD points to a ref.
4. **HEAD~n** first parents, **HEAD^2** second parent.
5. git commit = write-tree + commit-tree + update-ref.

PRACTICE NOW

Do exercises/ex09_commit_by_hand/: a real commit with plumbing only.